

# **Model-free methods: Monte-Carlo, Q-learning and Sarsa**



## **Solución de los ejercicios y parte opcional Taxi**

APRINS-GIA-FIB

Marc Delgado y Taisiia Prymak

10 / 10 / 2025

# 1. EJERCICIO 1.

Para empezar, se van a mostrar los resultados obtenidos a lo largo de las 10 ejecuciones de cada algoritmo y el resultado del cálculo de la varianza del *Average reward* :

## MONTECARLO:

| Experimentos | Policy | Average reward     |
|--------------|--------|--------------------|
| 1            |        | 0.7052800000000123 |
| 2            |        | 0.7541600000000021 |
| 3            |        | 0.7385600000000176 |
| 4            |        | 0.7452400000000169 |

|   |  |                    |
|---|--|--------------------|
| 5 |  | 0.7558000000000192 |
| 6 |  | 0.7466400000000208 |
| 7 |  | 0.6814400000000076 |
| 8 |  | 0.6753200000000058 |
| 9 |  | 0.7387600000000142 |

|          |  |                    |
|----------|--|--------------------|
| 10       |  | 0.7509200000000247 |
| Varianza |  | 0.0009221105956    |

### Q-LEARNING:

| Experimentos | Policy | Average reward     |
|--------------|--------|--------------------|
| 1            |        | 0.7106400000000103 |
| 2            |        | 0.735320000000015  |
| 3            |        | 0.7094800000000138 |
| 4            |        | 0.7384800000000157 |

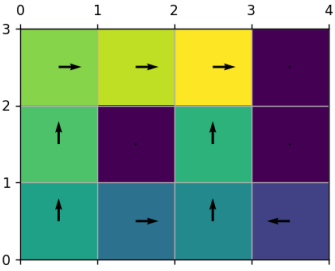
|   |  |                    |
|---|--|--------------------|
| 5 |  | 0.7154800000000121 |
| 6 |  | 0.7113600000000042 |
| 7 |  | 0.7514800000000181 |
| 8 |  | 0.7278400000000147 |
| 9 |  | 0.734240000000013  |

|          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                    |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|
| 10       | <p>A 4x4 grid world environment. The grid is indexed 0 to 3 on both axes. Actions are indicated by arrows: (0,0) up, (0,1) up, (0,2) right, (0,3) right, (1,0) right, (1,1) right, (1,2) right, (1,3) right, (2,0) up, (2,1) up, (2,2) up, (2,3) left, (3,0) left, (3,1) left, (3,2) left, (3,3) left. Rewards are indicated by colors: (0,0) green, (0,1) green, (0,2) yellow, (0,3) yellow, (1,0) green, (1,1) purple, (1,2) green, (1,3) purple, (2,0) green, (2,1) green, (2,2) green, (2,3) blue, (3,0) blue, (3,1) blue, (3,2) blue, (3,3) blue.</p> | 0.7334800000000015 |
| Varianza |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 0.0002052036444    |

### SARSA:

| Experimento | Policy                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Average reward      |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| 1           | <p>A 4x4 grid world environment. The grid is indexed 0 to 3 on both axes. Actions are indicated by arrows: (0,0) up, (0,1) up, (0,2) right, (0,3) right, (1,0) right, (1,1) right, (1,2) right, (1,3) right, (2,0) up, (2,1) up, (2,2) up, (2,3) left, (3,0) left, (3,1) left, (3,2) left, (3,3) left. Rewards are indicated by colors: (0,0) green, (0,1) green, (0,2) yellow, (0,3) yellow, (1,0) green, (1,1) purple, (1,2) green, (1,3) purple, (2,0) green, (2,1) green, (2,2) green, (2,3) blue, (3,0) blue, (3,1) blue, (3,2) blue, (3,3) blue.</p> | 0.75028000000000139 |
| 2           | <p>A 4x4 grid world environment. The grid is indexed 0 to 3 on both axes. Actions are indicated by arrows: (0,0) up, (0,1) up, (0,2) right, (0,3) right, (1,0) right, (1,1) right, (1,2) right, (1,3) right, (2,0) up, (2,1) up, (2,2) up, (2,3) left, (3,0) left, (3,1) left, (3,2) left, (3,3) left. Rewards are indicated by colors: (0,0) green, (0,1) green, (0,2) yellow, (0,3) yellow, (1,0) green, (1,1) purple, (1,2) green, (1,3) purple, (2,0) green, (2,1) green, (2,2) green, (2,3) blue, (3,0) blue, (3,1) blue, (3,2) blue, (3,3) blue.</p> | 0.74424000000000143 |
| 3           | <p>A 4x4 grid world environment. The grid is indexed 0 to 3 on both axes. Actions are indicated by arrows: (0,0) up, (0,1) up, (0,2) right, (0,3) right, (1,0) right, (1,1) right, (1,2) right, (1,3) right, (2,0) up, (2,1) up, (2,2) up, (2,3) left, (3,0) left, (3,1) left, (3,2) left, (3,3) left. Rewards are indicated by colors: (0,0) green, (0,1) green, (0,2) yellow, (0,3) yellow, (1,0) green, (1,1) purple, (1,2) green, (1,3) purple, (2,0) green, (2,1) green, (2,2) green, (2,3) blue, (3,0) blue, (3,1) blue, (3,2) blue, (3,3) blue.</p> | 0.72088000000000111 |
| 4           | <p>A 4x4 grid world environment. The grid is indexed 0 to 3 on both axes. Actions are indicated by arrows: (0,0) up, (0,1) up, (0,2) right, (0,3) right, (1,0) right, (1,1) right, (1,2) right, (1,3) right, (2,0) up, (2,1) up, (2,2) up, (2,3) left, (3,0) left, (3,1) left, (3,2) left, (3,3) left. Rewards are indicated by colors: (0,0) green, (0,1) green, (0,2) yellow, (0,3) yellow, (1,0) green, (1,1) purple, (1,2) green, (1,3) purple, (2,0) green, (2,1) green, (2,2) green, (2,3) blue, (3,0) blue, (3,1) blue, (3,2) blue, (3,3) blue.</p> | 0.72604000000000143 |

|   |  |                     |
|---|--|---------------------|
| 5 |  | 0.7085200000000064  |
| 6 |  | 0.7223600000000011  |
| 7 |  | 0.6921600000000075  |
| 8 |  | 0.7421600000000013  |
| 9 |  | 0.73744000000000121 |

|                 |                                                                                   |                    |
|-----------------|-----------------------------------------------------------------------------------|--------------------|
| 10              |  | 0.7211200000000105 |
| <b>Varianza</b> |                                                                                   | 0.0003139452444    |

En nuestros experimentos observamos que la política obtenida con **Monte Carlo, SARSA y Q-Learning** no siempre coincide exactamente entre ejecuciones, e incluso dentro de un mismo algoritmo pueden aparecer políticas distintas al repetir el entrenamiento.

Esto se debe a varios factores:

1. **Exploración elevada ( $\epsilon$  alto).**

Durante el entrenamiento se utiliza una política  $\epsilon$ -greedy, lo que significa que con cierta probabilidad (en nuestro caso 20%) el agente elige una acción aleatoria en lugar de la mejor según los valores actuales de Q. Este nivel de exploración es suficiente para garantizar que se visiten distintos estados y acciones, pero también introduce variabilidad: el agente puede aprender trayectorias distintas en función de las elecciones aleatorias realizadas.

2. **Tasa de aprendizaje alta.**

Un *learning rate* elevado puede hacer que las actualizaciones de los valores Q sean más bruscas y dependan más de las experiencias recientes. Esto acelera la convergencia inicial, pero también genera más oscilaciones y variación entre ejecuciones, ya que los Q-values se ajustan con fuerza a cada episodio concreto.

3. **Naturaleza estocástica del proceso.**

Tanto el muestreo de acciones (por la exploración) como la secuencia de episodios son procesos inherentemente aleatorios, el hecho de que las trayectorias dependan de decisiones exploratorias introduce aleatoriedad en los datos de entrenamiento a lo que, además se suma que el entorno no sea determinista.

Como consecuencia, **los tres algoritmos tienden a aproximarse a la política óptima**, pero no siempre aprenden exactamente la misma política tras un número finito de episodios. Las diferencias observadas no implican que uno de los algoritmos sea incorrecto, sino que reflejan la influencia de la exploración, del *learning rate* y del carácter estocástico del aprendizaje por refuerzo y el entorno.

**En cuanto a la varianza**, como es esperado, Monte Carlo es el algoritmo que tiene más varianza en comparación con el resto. Monte Carlo no introduce *bias* ya que no depende de la estimación sino que usa retornos reales, pero sí conlleva a una alta varianza al obtener los retornos al final del episodio, un episodio que es muy aleatorio a causa de la exploración. Como resultado Monte Carlo obtiene trayectorias muy diferentes entre sí, con retornos, también, muy diferentes.



Sin embargo, Q-learning usa una estimación promedio del retorno esto permite suavizar mucho más las recompensas atípicas obtenidas tras una exploración aleatoria. Además, en Q-learning, la actualización ocurre en cada paso de la trayectoria sin necesidad de acabar el episodio permitiendo corregir los valores Q con mucha más suavidad.

Por último, SARSA, tiene un valor de varianza muy parecido a Q-learning ya que también utiliza bootstrapping, pero es un valor ligeramente mayor debido a que los valores se calculan utilizando la acción  $a'$  que proviene de la política greedy (más aleatoriedad).

## 2. EJERCICIO 2.

Sí, las políticas siguen siendo razonables. Aunque Q-learning, SARSA y Monte Carlo no siempre convergen exactamente a la política óptima en cada ejecución, las políticas obtenidas suelen ser muy similares y conducen al agente hacia el objetivo con solo pequeñas variaciones en determinados estados.

Esto ocurre porque los tres algoritmos están diseñados para maximizar la recompensa acumulada a largo plazo y, pese a la estocasticidad introducida por la exploración ( $\epsilon$ -greedy) y por las actualizaciones de aprendizaje, capturan correctamente la estructura subyacente del entorno. Como consecuencia, las políticas aprendidas guían al agente por trayectorias eficientes, aunque la secuencia exacta de acciones pueda diferir ligeramente de la óptima.

En conclusión, incluso cuando no son estrictamente óptimas, las políticas obtenidas son razonables porque cumplen con el objetivo principal: alcanzar el estado meta de forma eficaz.

## 3. EJERCICIO 3.

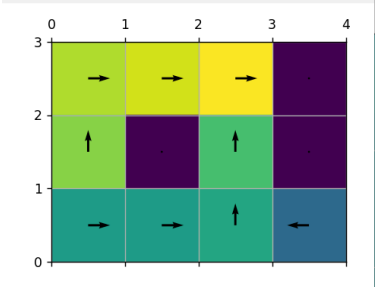
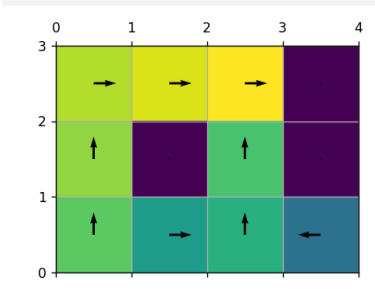
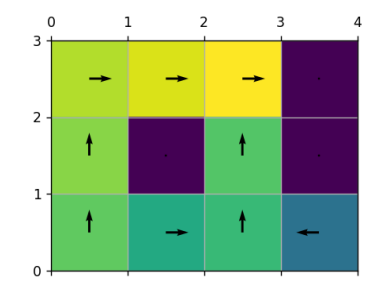
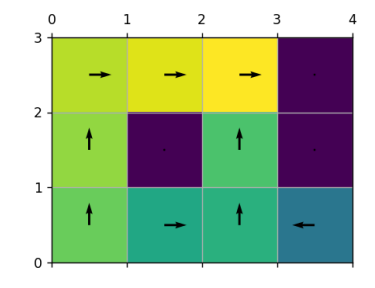
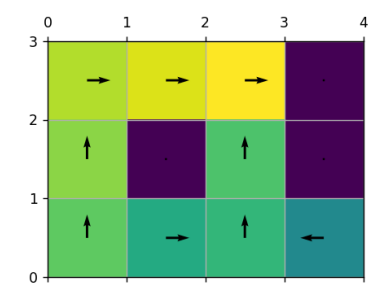
Para empezar, vamos a realizar los experimentos con  $\epsilon$  fijas. Como el valor con el que ya hemos experimentado es bastante alto (0.2), vamos a estudiar qué pasará si fijamos un  $\epsilon$  muy bajo.

Empezando por fijar una exploración muy baja de 0.01, se observa para todos los algoritmos a lo largo de diferentes ejecuciones, que los resultados de train son bastante buenos, aprende bastante rápido. Esto pasa porque se toman menos acciones aleatorias, lo cual reduce el riesgo de tomar acciones indebidas o caer en el estado terminal prohibido. Además se recibe, para todos los algoritmos, una recompensa de test bastante parecida a la que teníamos cuando la  $\epsilon$  era igual a 0.2. Los algoritmos, para este problema, consiguen converger a la misma política óptima, incluso con poca exploración, dando lugar a recompensas de test muy parecidas. Se podría decir que una exploración baja puede ser suficiente para el entorno *RussellGrid* que tenemos, ya que es pequeño y por eso es más fácil visitar las parejas *estado acción*.

A continuación, se van a mostrar los resultados de aplicar epsilon decay de 0.2 a 0.01:

### Q-LEARNING

| Experimento | Policy | Average reward |
|-------------|--------|----------------|
|-------------|--------|----------------|

|   |                                                                                     |                    |
|---|-------------------------------------------------------------------------------------|--------------------|
| 1 |    | 0.6880000000000088 |
| 2 |    | 0.7422800000000159 |
| 3 |   | 0.7296400000000133 |
| 4 |  | 0.7380400000000158 |
| 5 |  | 0.7140400000000133 |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                    |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|
| 6  | <p>A 4x4 grid world environment. The grid is indexed 0-3 on both axes. Actions are indicated by arrows: (0,0)→, (1,0)→, (2,0)↑, (3,0)←, (0,1)↑, (1,1)→, (2,1)→, (3,1)→, (0,2)↑, (1,2)→, (2,2)→, (3,2)→, (0,3)↑, (1,3)→, (2,3)→, (3,3)→. Rewards are indicated by colors: (0,0) light green, (1,0) light green, (2,0) light green, (3,0) dark purple, (0,1) light green, (1,1) dark purple, (2,1) light green, (3,1) dark purple, (0,2) light green, (1,2) dark purple, (2,2) light green, (3,2) dark purple, (0,3) light green, (1,3) dark purple, (2,3) light green, (3,3) dark purple.</p> | 0.690880000000007  |
| 7  | <p>A 4x4 grid world environment. The grid is indexed 0-3 on both axes. Actions are indicated by arrows: (0,0)↑, (1,0)→, (2,0)↑, (3,0)←, (0,1)↑, (1,1)→, (2,1)→, (3,1)→, (0,2)↑, (1,2)→, (2,2)→, (3,2)→, (0,3)↑, (1,3)→, (2,3)→, (3,3)→. Rewards are indicated by colors: (0,0) light green, (1,0) light green, (2,0) light green, (3,0) dark purple, (0,1) light green, (1,1) dark purple, (2,1) light green, (3,1) dark purple, (0,2) light green, (1,2) dark purple, (2,2) light green, (3,2) dark purple, (0,3) light green, (1,3) dark purple, (2,3) light green, (3,3) dark purple.</p> | 0.7043600000000101 |
| 8  | <p>A 4x4 grid world environment. The grid is indexed 0-3 on both axes. Actions are indicated by arrows: (0,0)↑, (1,0)→, (2,0)↑, (3,0)←, (0,1)↑, (1,1)→, (2,1)→, (3,1)→, (0,2)↑, (1,2)→, (2,2)→, (3,2)→, (0,3)↑, (1,3)→, (2,3)→, (3,3)→. Rewards are indicated by colors: (0,0) light green, (1,0) light green, (2,0) light green, (3,0) dark purple, (0,1) light green, (1,1) dark purple, (2,1) light green, (3,1) dark purple, (0,2) light green, (1,2) dark purple, (2,2) light green, (3,2) dark purple, (0,3) light green, (1,3) dark purple, (2,3) light green, (3,3) dark purple.</p> | 0.7269200000000112 |
| 9  | <p>A 4x4 grid world environment. The grid is indexed 0-3 on both axes. Actions are indicated by arrows: (0,0)↑, (1,0)→, (2,0)↑, (3,0)←, (0,1)↑, (1,1)→, (2,1)→, (3,1)→, (0,2)↑, (1,2)→, (2,2)→, (3,2)→, (0,3)↑, (1,3)→, (2,3)→, (3,3)→. Rewards are indicated by colors: (0,0) light green, (1,0) light green, (2,0) light green, (3,0) dark purple, (0,1) light green, (1,1) dark purple, (2,1) light green, (3,1) dark purple, (0,2) light green, (1,2) dark purple, (2,2) light green, (3,2) dark purple, (0,3) light green, (1,3) dark purple, (2,3) light green, (3,3) dark purple.</p> | 0.7210800000000157 |
| 10 | <p>A 4x4 grid world environment. The grid is indexed 0-3 on both axes. Actions are indicated by arrows: (0,0)↑, (1,0)→, (2,0)↑, (3,0)←, (0,1)↑, (1,1)→, (2,1)→, (3,1)→, (0,2)↑, (1,2)→, (2,2)→, (3,2)→, (0,3)↑, (1,3)→, (2,3)→, (3,3)→. Rewards are indicated by colors: (0,0) light green, (1,0) light green, (2,0) light green, (3,0) dark purple, (0,1) light green, (1,1) dark purple, (2,1) light green, (3,1) dark purple, (0,2) light green, (1,2) dark purple, (2,2) light green, (3,2) dark purple, (0,3) light green, (1,3) dark purple, (2,3) light green, (3,3) dark purple.</p> | 0.7110400000000111 |

### SARSA:

| Experimento | Policy | Average reward |
|-------------|--------|----------------|
|-------------|--------|----------------|

|   |  |                    |
|---|--|--------------------|
| 1 |  | 0.7297200000000126 |
| 2 |  | 0.7006800000000123 |
| 3 |  | 0.7094000000000089 |
| 4 |  | 0.7536000000000168 |
| 5 |  | 0.7224800000000121 |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                    |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|
| 6  | <p>A 4x4 grid world environment. The grid is indexed from 0 to 4 on both axes. The top row (y=3) has actions: right, right, right, and a wall. The second row (y=2) has actions: up, a wall, up, and a wall. The third row (y=1) has actions: up, right, up, and left. The bottom row (y=0) has actions: up, right, up, and left. The grid is colored: (0,3) is light green, (1,3) is yellow, (2,3) is yellow, (3,3) is dark purple. (0,2) is light green, (1,2) is dark purple, (2,2) is light green, (3,2) is dark purple. (0,1) is light green, (1,1) is teal, (2,1) is light green, (3,1) is dark blue. (0,0) is light green, (1,0) is teal, (2,0) is light green, (3,0) is dark blue.</p> | 0.7190400000000154 |
| 7  | <p>A 4x4 grid world environment. The grid is indexed from 0 to 4 on both axes. The top row (y=3) has actions: right, right, right, and a wall. The second row (y=2) has actions: up, a wall, up, and a wall. The third row (y=1) has actions: up, right, up, and left. The bottom row (y=0) has actions: up, right, up, and left. The grid is colored: (0,3) is light green, (1,3) is yellow, (2,3) is yellow, (3,3) is dark purple. (0,2) is light green, (1,2) is dark purple, (2,2) is light green, (3,2) is dark purple. (0,1) is light green, (1,1) is teal, (2,1) is light green, (3,1) is dark blue. (0,0) is light green, (1,0) is teal, (2,0) is light green, (3,0) is dark blue.</p> | 0.7440000000000164 |
| 8  | <p>A 4x4 grid world environment. The grid is indexed from 0 to 4 on both axes. The top row (y=3) has actions: right, right, right, and a wall. The second row (y=2) has actions: up, a wall, up, and a wall. The third row (y=1) has actions: up, right, up, and left. The bottom row (y=0) has actions: up, right, up, and left. The grid is colored: (0,3) is light green, (1,3) is yellow, (2,3) is yellow, (3,3) is dark purple. (0,2) is light green, (1,2) is dark purple, (2,2) is light green, (3,2) is dark purple. (0,1) is light green, (1,1) is teal, (2,1) is light green, (3,1) is dark blue. (0,0) is light green, (1,0) is teal, (2,0) is light green, (3,0) is dark blue.</p> | 0.6943200000000099 |
| 9  | <p>A 4x4 grid world environment. The grid is indexed from 0 to 4 on both axes. The top row (y=3) has actions: right, right, right, and a wall. The second row (y=2) has actions: up, a wall, up, and a wall. The third row (y=1) has actions: up, right, up, and left. The bottom row (y=0) has actions: up, right, up, and left. The grid is colored: (0,3) is light green, (1,3) is yellow, (2,3) is yellow, (3,3) is dark purple. (0,2) is light green, (1,2) is dark purple, (2,2) is light green, (3,2) is dark purple. (0,1) is light green, (1,1) is teal, (2,1) is light green, (3,1) is dark blue. (0,0) is light green, (1,0) is teal, (2,0) is light green, (3,0) is dark blue.</p> | 0.7521200000000147 |
| 10 | <p>A 4x4 grid world environment. The grid is indexed from 0 to 4 on both axes. The top row (y=3) has actions: right, right, right, and a wall. The second row (y=2) has actions: up, a wall, up, and a wall. The third row (y=1) has actions: up, right, up, and left. The bottom row (y=0) has actions: up, right, up, and left. The grid is colored: (0,3) is light green, (1,3) is yellow, (2,3) is yellow, (3,3) is dark purple. (0,2) is light green, (1,2) is dark purple, (2,2) is light green, (3,2) is dark purple. (0,1) is light green, (1,1) is teal, (2,1) is light green, (3,1) is dark blue. (0,0) is light green, (1,0) is teal, (2,0) is light green, (3,0) is dark blue.</p> | 0.7104800000000098 |

#### MONTECARLO:

| Experimento | Policy | Average reward |
|-------------|--------|----------------|
|-------------|--------|----------------|

|   |  |                    |
|---|--|--------------------|
| 1 |  | 0.7500400000000137 |
| 2 |  | 0.7242800000000157 |
| 3 |  | 0.7685200000000211 |
| 4 |  | 0.7595600000000274 |
| 5 |  | 0.7148000000000115 |

|    |  |                     |
|----|--|---------------------|
| 6  |  | 0.7552400000000197  |
| 7  |  | 0.6982000000000078  |
| 8  |  | 0.7284000000000093  |
| 9  |  | 0.73836000000000249 |
| 10 |  | 0.7495200000000157  |

### Resultados observados.

Tras entrenar con  $\epsilon$  **decreciente** de 0.2 a 0.01 durante 10 000 episodios:

- **Q-learning:** las 10 ejecuciones convergieron a la política óptima, con algo de variabilidad en cómo son las políticas resultantes.
- **SARSA:** las 10 ejecuciones convergieron a la misma política óptima.
- **Monte-Carlo:** se observaron políticas más diversas, con mayor variabilidad entre ejecuciones.

Que Q-learning y SARSA tiendan a coincidir más en la política final es esperable, pues ambos algoritmos utilizan **bootstrapping**: actualizan los valores Q en cada paso a partir de las estimaciones actuales de la tabla. Con un  $\epsilon$  que va decayendo, el agente explora al inicio y luego explota cada vez más, lo que les permite refinar progresivamente la política hasta estabilizarse cerca del óptimo.

En cambio, Monte-Carlo sigue mostrando **mayor variabilidad** por varias razones:

1. **Actualización episódica completa.**

Monte-Carlo no aprovecha cada transición, sino que espera a terminar el episodio para calcular el retorno total. Esto implica que el aprendizaje es más lento y cada episodio pesa mucho, sobre todo si hay alta varianza en las recompensas acumuladas.

2. **Alta varianza inherente.**

Como el retorno depende de toda la secuencia de estados y acciones, pequeñas diferencias en la exploración inicial (cuando  $\epsilon$  aún es relativamente alto) pueden producir trayectorias muy distintas. Esto amplifica la dispersión entre ejecuciones, incluso si  $\epsilon$  decrece después.

3. **Menor eficiencia muestral.**

Mientras que Q-learning y SARSA reutilizan información de cada paso, Monte-Carlo necesita muchos más episodios para estabilizar los valores Q. Con un número limitado de episodios (10000), es normal que quede variabilidad residual en las políticas finales.

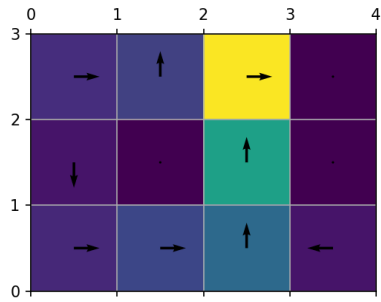
## 4. EJERCICIO 4.

Para empezar, haremos los experimentos con una tasa de aprendizaje de 0.0001:

- **Monte Carlo:** durante el entrenamiento podemos apreciar una mejora de la recompensa obtenida a medida que avanza el entrenamiento. Sin embargo, la mejora es muy lenta por lo que las recompensas durante el aprendizaje no suelen superar el valor de 0.58. MC actualiza con menor frecuencia (solo al final del episodio) por lo que necesita pasos más grandes para converger en tiempo razonable. Como, de esta manera, tarda mucho en converger puede alcanzar el máximo número de episodios en el punto donde aún no se ha llegado a



A horizontal number line with tick marks at 0, 1, 2, 3, and 4. The segment between 2 and 3 is highlighted in yellow.



El hecho de que el algoritmo aún no ha convergido, también se puede ver en la matriz  $V$  resultante. Por ejemplo, vemos en la posición (0,0) una estimación de retorno futuro muy baja pero en realidad solo debería de dar 3 pasos hasta alcanzar la meta. Una vez más se concluye que el algoritmo no ha tenido suficiente tiempo para converger. Se necesita una tasa de aprendizaje más alta para converger antes de que se alcance el número máximo de episodios. Por último, hay que destacar que en este caso la varianza parece aumentar vista la mayor variabilidad entre ejecuciones, lo que resulta a una mayor sensibilidad a las acciones aleatorias y a la inestabilidad del entorno.

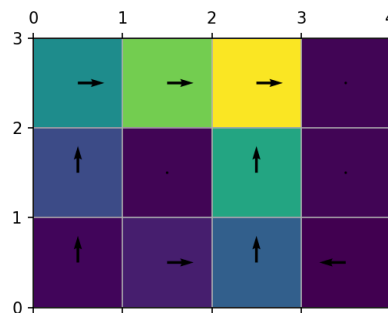
- **Q-learning:** para este algoritmo se observan retornos de *train* incluso peores que con el algoritmo MC. Ambas implementaciones usan la misma regla  $Q \leftarrow Q + \alpha (target - Q)$ , pero si  $\alpha = 0.0001$ , cada actualización cambia  $Q$  en una fracción minúscula del error. En Q-learning el *target* es  $r + \gamma \max Q(next)$ , al inicio  $Q(next) \approx 0$  y  $r$  suele ser pequeño, por lo que el error por paso es pequeño y con  $\alpha$  tan pequeño, ese pequeño error apenas mueve  $Q$ . Mientras que MC utiliza el retorno de todo el episodio para actualizar los valores  $Q$ , lo cual en magnitud es mucho mayor que la actualización de Q-learning.
- **SARSA:** obtiene unos resultados casi idénticos a Q-learning, lo cual es comprensible porque ambos usan bootstrapping. Pero, muy raras veces, podemos encontrarnos con un resultado ligeramente mejor que los que se obtienen con Q-learning y esto puede ser debido a que SARSA es un algoritmo on-policy y tiene en cuenta la política real que se está siguiendo. En SARSA, la política aprendida y la ejecutada son la misma, los valores  $Q$  mejoran exactamente en la dirección de aumentar esas recompensas. En Q-learning, los  $Q$  se ajustan hacia una política distinta (la greedy), así que los valores pueden mejorar para la política greedy, pero eso no se refleja aún en las recompensas reales, porque la política ejecutada ( $\epsilon$ -greedy) no coincide con ella.

A continuación, se van a discutir los resultados obtenidos con una tasa de aprendizaje de 0.001:

- Monte Carlo: ahora un poco más frecuentemente las ejecuciones tienen tiempo suficiente para converger a la política óptima, pero sigue persistiendo el mismo problema que hemos comentado al estudiar el caso anterior.
- **Q-learning:** Al aumentar la tasa de aprendizaje a 0.001, Q-learning siempre consigue converger a la política óptima, en comparación con Monte Carlo que no muestra una mejora tan marcada. Esto se debe a que Q-learning realiza

actualizaciones más frecuentes y con una señal de error de menor varianza, por lo que puede aprovechar un  $\alpha$  mayor sin volverse inestable. En cambio, Monte Carlo utiliza el retorno total del episodio, de alta varianza, de modo que un  $\alpha$  más alto amplifica la inestabilidad y las oscilaciones en los valores Q. Como consecuencia, Q-learning logra converger más rápido y de forma más consistente hacia la política óptima, mientras que Monte Carlo sigue mostrando dificultades para estabilizarse en el número de episodios disponible.

- **SARSA:** consigue alcanzar la política óptima mostrando un comportamiento muy parecido a Q-learning, pero es importante comentar que, para estos dos algoritmos, la mayoría de veces los valores Q encontrados tienden a ser mucho menos optimistas que cuando teníamos un  $lr$  de 0.01, por ejemplo:



Esto indica que se estima una recompensa menor para un estado que la que se estima cuando teníamos un  $lr$  de 0.01 indicando que deberíamos de hacer más iteraciones para obtener una mejor estimación de los valores. Es decir, un  $lr$  de 0.001 la mayoría de veces nos permite llegar a convergencia dentro de los episodios que hemos definido, pero al ser la actualización muy lenta obtenemos una estimación un poco subestimada.

## 5. EJERCICIO 5.

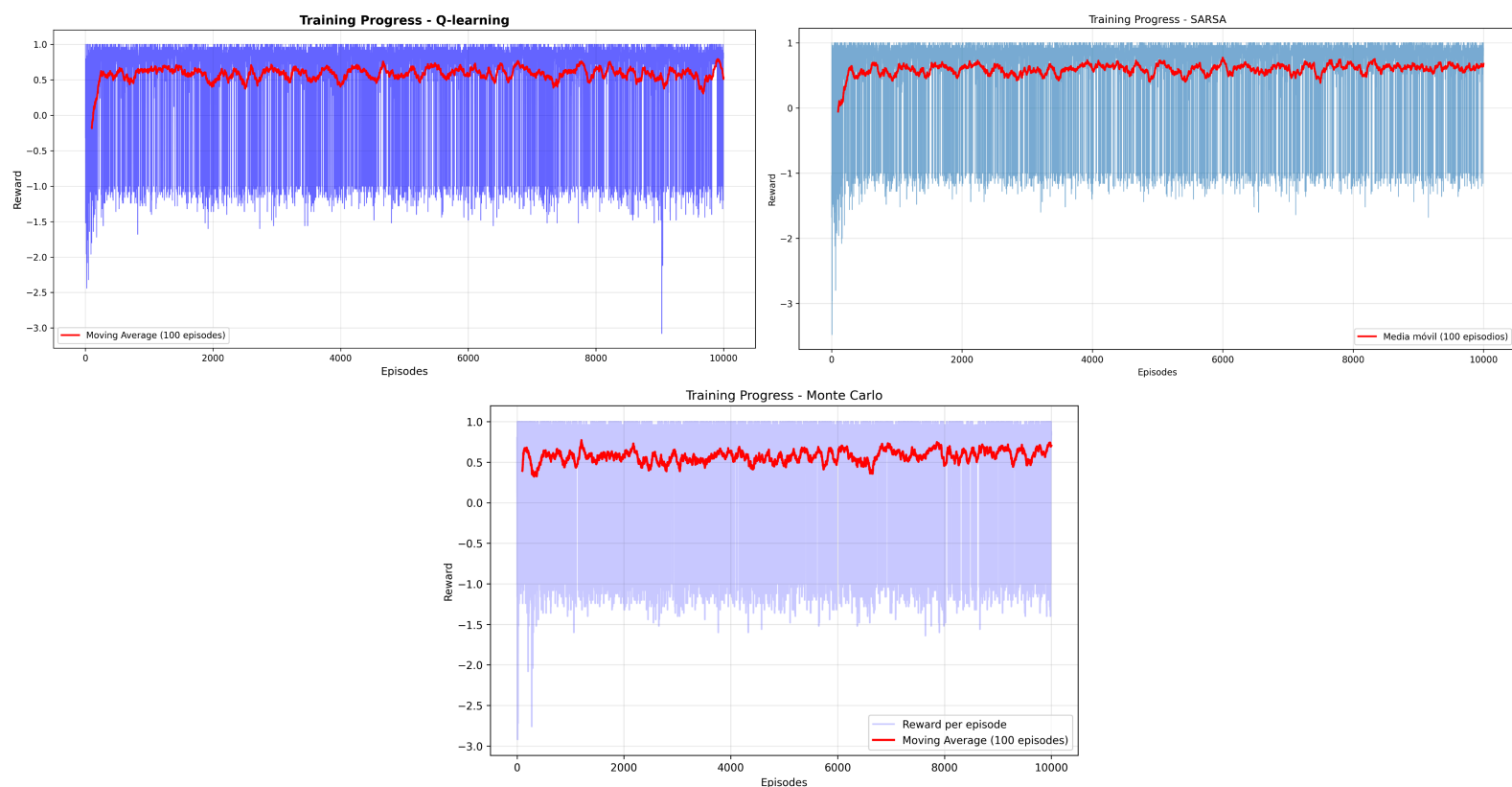
Para comparar el rendimiento durante el entrenamiento y durante el test dejamos la siguiente ejecución como ejemplo (para todos los algoritmos fijamos el learning rate a 0,01 y epsilon a 0,2):

| Número de Episodios | Total Reward MC    | Total Reward Q-learning | Total Reward SARSA |
|---------------------|--------------------|-------------------------|--------------------|
| 500                 | 0.3883199999999999 | 0.3760799999999999      | 0.4468800000000000 |
| 1000                | 0.4245599999999999 | 0.5378399999999999      | 0.5948800000000000 |
| 1500                | 0.4828799999999999 | 0.5579199999999999      | 0.6295999999999999 |
| 2000                | 0.4731999999999999 | 0.5463199999999999      | 0.6332799999999998 |
| 2500                | 0.4373599999999999 | 0.6148799999999999      | 0.6387199999999999 |
| 3000                | 0.5079199999999998 | 0.6174399999999999      | 0.6210399999999999 |

|                            |                   |                   |                   |
|----------------------------|-------------------|-------------------|-------------------|
| 3500                       | 0.441280000000000 | 0.610479999999999 | 0.597280000000000 |
| 4000                       | 0.501119999999999 | 0.602879999999999 | 0.621839999999999 |
| 4500                       | 0.426719999999999 | 0.574399999999999 | 0.595679999999999 |
| 5000                       | 0.490479999999999 | 0.578879999999999 | 0.590799999999999 |
| 5500                       | 0.42944           | 0.641119999999999 | 0.646239999999999 |
| 6000                       | 0.521199999999999 | 0.589679999999999 | 0.599279999999999 |
| 6500                       | 0.460479999999999 | 0.604319999999999 | 0.574880000000000 |
| 7000                       | 0.506719999999999 | 0.571919999999999 | 0.609119999999999 |
| 7500                       | 0.469440000000000 | 0.607759999999999 | 0.626319999999999 |
| 8000                       | 0.473200000000000 | 0.632639999999999 | 0.59968           |
| 8500                       | 0.404879999999999 | 0.573679999999999 | 0.632719999999999 |
| 9000                       | 0.578159999999999 | 0.577999999999999 | 0.605920000000000 |
| 9500                       | 0.552639999999999 | 0.533759999999999 | 0.641119999999999 |
| 10000                      | 0.586799999999999 | 0.650799999999999 | 0.667919999999999 |
| <b>Avg. testing reward</b> | 0.726560000000001 | 0.745800000000001 | 0.725920000000001 |

Se puede ver que durante el entrenamiento es MC quien obtiene recompensas más pequeñas más a menudo. Esto ocurre porque los valores Q se actualizan solo al final de cada episodio, entonces la señal de mejora tarda más en llegar, al contrario de Q-learning y SARSA que antes de terminar el episodio cambian la política y por lo tanto pueden mejorar la recompensa final de cada episodio creando menos varianza.

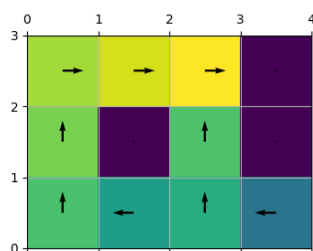
En general, Q-learning y SARSA tienen un comportamiento bastante parecido, salvo que Q-learning suele obtener resultados más bajos más veces que SARSA (que casi nunca baja de una recompensa de 0.59) esto se debe a que Q-learning actualiza la matriz Q tomando la acción greedy  $\text{Max } a' (Q(s', a'))$  y por tanto puede tomar decisiones más arriesgadas y por culpa de la exploración acabar en el estado terminal prohibido.



Por último, hay que decir que el rendimiento en el test es muy similar. Mirando este ejemplo, no podríamos afirmar que Q-learning siempre tiene resultados ligeramente mejores debido a que, como ya hemos visto, puede tener algo de varianza.

## 6. Ejercicio 6.

En el lab anterior encontrábamos siempre la política óptima ya que disponíamos del modelo del entorno, es decir, teníamos las probabilidades de transición estado-acción. Esta es la política a la que se converge:



Trás muchos experimentos, tal y como se puede ver en los apartados anteriores, el resultado que obtenemos es que el algoritmo Monte Carlo algunas veces consigue la misma política óptima que al algoritmo policy iteration model based y donde la matriz  $V$  sale bastante parecida (como se puede ver por los colores del greed). Pero, además, algunas veces MC decide tomar el camino más corto, también el más arriesgado, donde cambia la acción en la posición (2, 1) en el grid, esto es debido a la alta varianza.

Por otro lado, vemos que SARSA y Q-learning siempre convergen en el camino más corto. En este caso, el camino más corto es también el arriesgado lo que se puede ver por la matriz  $V$  que estima las recompensas esperadas. Cuando miramos las matrices  $V$  de Q-learning y SARSA suelen elegir la dirección más arriesgada pero más rápida para la posición (2, 1), hacia la derecha aunque la recompensa esperada para ese estado se vuelve mucho menor (es menor por que existe más riesgo de caer en el estado terminal prohibido) que en el caso de la política que decide ir a la izquierda.

Este comportamiento puede ser debido a que MC espera hasta el final del episodio y usa el retorno real completo. No hace suposiciones sobre valores futuros, simplemente promedia retornos observados, y no depende de estimaciones de otros estados (no hay bootstrapping), por lo que el ruido no se propagan entre estados. Esto lleva a que MC no tenga bias, mientras que los otros dos algoritmos sí.

## 7. Ejercicio 7.

Trás múltiples ejecuciones se ve que SARSA siempre suele converger a la política que el Algoritmo Policy Iteration (visto en el laboratorio 2) que se ha mencionado en el apartado anterior. Esta política toma decisiones más conservadoras por que intenta mantenerse lejos de la celda terminal prohibida y por lo tanto en la posición (2,1) del grid decide darle la vuelta a la columna.

Por otro lado el algoritmo de Q-learning siempre suele converger a la política más arriesgada, pero la que sigue el camino más corto, por esta razón decide tomar la acción hacia la derecha en la celda (2, 1).

Este comportamiento de estos dos algoritmos es una de sus diferencias fundamentales. Q-learning es un algoritmo off-policy y aprende el valor de la política óptima, la política greedy, pero en realidad no la sigue y ignora completamente la exploración que en realidad está siguiendo. Como resultado elige caminos más cortos, que en este caso es un óptimo arriesgado. Mientras que SARSA, que es un algoritmo on-policy, sí que aprende la misma política que está siguiendo (incluyendo exploración aleatoria) y como resultado evita zonas peligrosas y elige políticas más conservadoras.

Este patrón, también, se puede ver durante el entrenamiento. Durante el entrenamiento Q-learning suele obtener muchas recompensas muy bajas (valores 0.3 - 0.4), mientras que SARSA no obtiene tantos (valores de 0.5 a 0.71 hacia el final de entrenamiento). Sin embargo, como se ha establecido un entorno completamente determinista y durante el test no se usa exploración, Q-learning siempre obtiene resultados mejores que SARSA (ejemplo de ejecución: Avg. Reward in testing de SARSA es 0.8712800000000298 mientras que para Q-learning es 0.9149600000000225). Esto es lo esperado porque Q-learning al ir por el camino corto hace menos pasos por lo que acumula menos penalización por cada celda común (penalización de -0.4).

## 8. Ejercicio 8.

Después de múltiples ejecuciones se puede ver que Montecarlo es más parecido a SARSA ya que, en este caso, siempre converge la política óptima más conservadora. Esto se debe

a que el algoritmo de MC es on-policy así que tiene en cuenta la exploración usada en la política, no se queda con el camino más corto ya que la penalización del estado prohibido (donde tiene probabilidad de caer debido a la exploración) afectó mucho al promedio que hace el algoritmo. También, por esta razón obtiene un resultado de test muy parecido al de SARSA en el apartado anterior, mientras que durante casi todo el entrenamiento MC parece obtener unos resultados mucho mejores y estables, esto puede pasar porque se espera a terminar el episodio para actualizar la política, mientras que SARSA hace actualizaciones durante el episodio por lo que puede generar resultados más inestables más a menudo.

## 9. Ejercicio 9.

Q-learning sigue consiguiendo aprender la política óptima porque es un algoritmo *off-policy*, no tiene en cuenta que política se está siguiendo porque siempre aprende la política greedy con  $\max Q(s', a')$  por eso los valores de entrenamiento son muy malos (se sigue siempre una política aleatoria) pero los valores Q consiguen converger igualmente. Lo que se ve a cabo de muchas ejecuciones, es que el algoritmo termina quedándose en la política conservadora (no la del camino más corto como hemos visto en apartados anteriores) esto puede ser debido a que, en este caso, el algoritmo de Q-learning necesitaría más iteraciones de entrenamiento porque está visitando cada estado acción de forma esporádica.

## 10. PARTE OPCIONAL(TAXI)

Durante la ejecución del algoritmo Q-learning en el entorno Taxi-v3 de Gymnasium, el agente fue entrenado durante 10.000 episodios con una tasa de aprendizaje de 0.01, un factor de descuento de 0.999 y un valor de epsilon de 0.2. El objetivo del experimento era que el taxi aprendiera a recoger al pasajero y dejarlo en el destino correcto minimizando las penalizaciones que se obtienen por realizar movimientos innecesarios o acciones incorrectas.

Al inicio del entrenamiento el agente actuaba de forma casi aleatoria debido a la alta exploración, lo que provocaba recompensas medias muy bajas, alrededor de -300 puntos en los primeros episodios. Sin embargo, a medida que avanzaba el aprendizaje, las recompensas fueron mejorando progresivamente hasta estabilizarse alrededor de -13 al final del entrenamiento.

Durante la fase de prueba sin exploración, el valor medio de recompensa obtenido fue de -73.04 tras 1000 episodios. Aunque sigue siendo negativo, este resultado es esperable en el entorno Taxi, ya que cada movimiento tiene un coste y las únicas acciones con recompensa positiva son las de recoger y dejar correctamente al pasajero. El valor negativo simplemente indica que el agente realiza algunos movimientos intermedios antes de completar la tarea, pero aun así ha aprendido una secuencia razonablemente buena de acciones.

En general, los resultados muestran que el agente logró aprender una política estable y funcional mediante el uso de Q-learning. La tendencia descendente en la magnitud de las recompensas negativas confirma que el modelo fue mejorando su rendimiento con el tiempo.

En conclusión, la ejecución demuestra que el algoritmo Q-learning aplicado al entorno Taxi-v3 es capaz de aprender una política eficaz a partir de la experiencia, mejorando gradualmente su rendimiento.

```
PS C:\Users\marcd\Desktop\APRNS\PRACTICA\LAB3> python3 .\qlearning_taxi.py
Training...
Episode 500 Average Reward: -309.06 Epsilon = 0.2 lr = 0.01
Episode 1000 Average Reward: -270.32 Epsilon = 0.2 lr = 0.01
Episode 1500 Average Reward: -244.19 Epsilon = 0.2 lr = 0.01
Episode 2000 Average Reward: -213.34 Epsilon = 0.2 lr = 0.01
Episode 2500 Average Reward: -187.89 Epsilon = 0.2 lr = 0.01
Episode 3000 Average Reward: -163.70 Epsilon = 0.2 lr = 0.01
Episode 3500 Average Reward: -135.89 Epsilon = 0.2 lr = 0.01
Episode 4000 Average Reward: -114.13 Epsilon = 0.2 lr = 0.01
Episode 4500 Average Reward: -99.49 Epsilon = 0.2 lr = 0.01
Episode 5000 Average Reward: -82.52 Epsilon = 0.2 lr = 0.01
Episode 5500 Average Reward: -66.93 Epsilon = 0.2 lr = 0.01
Episode 6000 Average Reward: -56.28 Epsilon = 0.2 lr = 0.01
Episode 6500 Average Reward: -41.56 Epsilon = 0.2 lr = 0.01
Episode 7000 Average Reward: -39.23 Epsilon = 0.2 lr = 0.01
Episode 7500 Average Reward: -29.05 Epsilon = 0.2 lr = 0.01
Episode 8000 Average Reward: -24.19 Epsilon = 0.2 lr = 0.01
Episode 8500 Average Reward: -20.52 Epsilon = 0.2 lr = 0.01
Episode 9000 Average Reward: -18.10 Epsilon = 0.2 lr = 0.01
Episode 9500 Average Reward: -13.93 Epsilon = 0.2 lr = 0.01
Episode 10000 Average Reward: -13.77 Epsilon = 0.2 lr = 0.01
End training...
Visualizing episodes...
Collecting reward of the policy while testing (no exploration)...
Average reward of the policy while testing: -73.043
```